

Module 5

Advanced UVM Concepts

Learning objectives

- Master sequences, coverage, configuration, and virtual sequences

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Advanced Sequences"]

T2["UVM Coverage Models"]

T1 --> T2

T3["Complex Configuration Object"]

T2 --> T3

Master sequences, coverage, configuration, and virtual sequences

Overview

- This module covers advanced UVM concepts including virtual sequences, coverage models, complex conf...

Design architecture

RTL / block architecture

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph mc["multi_channel.v"]

CH0[channel 0]

CH1[channel 1]

CH2[channel N]

end

Module 5: DUT structure and key interfaces (see common_dut/rtl/).

Verification environment architecture

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart TB

VS[virtual sequencer] --> A0[agent 0]

VS --> A1[agent 1]

A0 --> DUT[(multi_channel)]

A1 --> DUT

MON[monitors] --> COV[coverage]

Module 5: UVM layers, agents, and checkers for this phase.

1. Advanced DUT architecture

- ``module5/dut/advanced/multi_channel.v`` — multiple logical channels for coordination labs
- Designed for virtual sequences and multi-stream scoreboarding
- Register abstraction layer examples align with memory-mapped control/status

2. Advanced UVM environment architecture

- Virtual sequencers coordinate multiple agents on different channels
- Coverage collectors sample transaction fields and cross coverage
- Callbacks extend driver/monitor without subclass explosion
- Configuration objects centralize knobs (active/passive, timeouts, enable flags)

3. Register model layer

- RAL-style maps: registers, fields, access policies (RW, RO, WO)
- Adapter links bus transactions to register reads/writes in tests
- Predictor/update paths keep mirror model consistent with DUT

Verification & testing methods

Testing and closure flow

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

VS[virtual sequence] --> COV[sample coverage]

COV --> G{Goals met?}

G -->|yes| DONE[sign-off]

G -->|no| MORE[add tests]

MORE --> VS

Module 5: how tests, checks, and metrics close verification goals.

1. Coverage-driven verification

- Functional coverage on transaction types, channel IDs, and corner field values
- Coverage goals gate regression sign-off in advanced flows
- Examples under ``module5/examples/coverage/`` show sampling hooks

2. Virtual sequences and configuration tests

- Virtual sequences start child sequences on multiple sequencers in order
- Config tests prove ConfigDB + config object overrides change behavior
- Callback tests inject errors or delays to validate robustness

3. Closure

- ``./scripts/module5.sh --check``; register model and callback demos in EXAMPLES.md
- Assessment: virtual sequences, coverage, callbacks, configuration, register model

Syllabus topics

1. Advanced Sequences (1/4)

- Virtual Sequences
- What are virtual sequences?
- Virtual sequence purpose
- Multiple sequencer coordination
- Virtual sequence implementation
- Sequence Libraries

1. Advanced Sequences (2/4)

- Base sequence classes
- Derived sequences
- Sequence reuse patterns
- Sequence organization
- Sequence Arbitration
- Sequencer arbitration

1. Advanced Sequences (3/4)

- Priority mechanisms
- Lock and grab
- Sequence coordination
- Layered Sequences
- High-level sequences
- Low-level sequences

1. Advanced Sequences (4/4)

- Sequence composition
- Protocol layering

2. UVM Coverage Models (1/4)

- Coverage Overview
- What is coverage?
- Coverage types
- Coverage goals
- Coverage metrics
- Functional Coverage

2. UVM Coverage Models (2/4)

- Coverage models
- Coverage groups
- Coverpoints
- Coverage bins
- Coverage Implementation
- Coverage class structure

2. UVM Coverage Models (3/4)

- Coverage sampling
- Coverage analysis
- Coverage reporting
- Coverage Patterns
- Transaction coverage
- Protocol coverage

2. UVM Coverage Models (4/4)

- State coverage
- Cross coverage

3. Complex Configuration Objects (1/4)

- Configuration Objects
- Configuration class design
- Configuration fields
- Configuration methods
- Configuration validation
- Configuration Hierarchy

3. Complex Configuration Objects (2/4)

- Hierarchical configuration
- Configuration inheritance
- Configuration override
- Configuration patterns
- Resource Database
- Resource database usage

3. Complex Configuration Objects (3/4)

- Resource types
- Resource lookup
- Resource management
- Configuration Callbacks
- Configuration callbacks
- Pre/post callbacks

3. Complex Configuration Objects (4/4)

- Callback registration
- Callback execution

4. UVM Callbacks (1/4)

- Callback Overview
- What are callbacks?
- Callback purpose
- Callback types
- Callback benefits
- Callback Implementation

4. UVM Callbacks (2/4)

- Callback class definition
- Callback registration
- Callback execution
- Callback patterns
- Pre/Post Callbacks
- Pre-callbacks

4. UVM Callbacks (3/4)

- Post-callbacks
- Callback ordering
- Callback control
- Callback Use Cases
- Driver callbacks
- Monitor callbacks

4. UVM Callbacks (4/4)

- Scoreboard callbacks
- Test callbacks

5. UVM Register Model (Advanced) (1/5)

- Register Model Overview
- Register model purpose
- Register model structure
- Register model benefits
- Register model components
- Register Model Components

5. UVM Register Model (Advanced) (2/5)

- ``uvm_reg_block`` - Register blocks
- ``uvm_reg`` - Registers
- ``uvm_reg_field`` - Register fields
- ``uvm_reg_map`` - Address maps
- Register Operations
- Register read

5. UVM Register Model (Advanced) (3/5)

- Register write
- Register peek/poke
- Register update
- Register Sequences
- Register access sequences
- Register test sequences

5. UVM Register Model (Advanced) (4/5)

- Register model integration
- Register predictor
- Backdoor Access
- Backdoor read/write
- Backdoor vs frontdoor
- Backdoor use cases

5. UVM Register Model (Advanced) (5/5)

- Backdoor implementation

6. Virtual Sequences and Virtual Sequencers (1/3)

- Virtual Sequencer
- Virtual sequencer purpose
- Virtual sequencer structure
- Multiple sequencer references
- Virtual sequencer implementation
- Virtual Sequence Coordination

6. Virtual Sequences and Virtual Sequencers (2/3)

- Coordinating multiple sequencers
- Parallel sequence execution
- Sequence synchronization
- Sequence coordination patterns
- Virtual Sequence Patterns
- Master-slave coordination

6. Virtual Sequences and Virtual Sequencers (3/3)

- Multi-channel coordination
- Protocol coordination
- Test coordination

7. Coverage Analysis and Closure (1/3)

- Coverage Analysis
- Coverage collection
- Coverage reporting
- Coverage gaps
- Coverage analysis tools
- Coverage Closure

7. Coverage Analysis and Closure (2/3)

- Coverage goals
- Coverage strategies
- Coverage improvement
- Coverage metrics
- Coverage Patterns
- Functional coverage

7. Coverage Analysis and Closure (3/3)

- Code coverage
- Assertion coverage
- Coverage correlation

8. Advanced Configuration Patterns (1/2)

- Configuration Strategies
 - Top-down configuration
 - Bottom-up configuration
 - Mixed configuration
- Configuration best practices
- Dynamic Configuration

8. Advanced Configuration Patterns (2/2)

- Runtime configuration
- Configuration updates
- Configuration validation
- Configuration debugging

9. Performance Optimization (1/2)

- Testbench Performance
- Performance bottlenecks
- Optimization strategies
- Memory optimization
- Simulation speed
- Sequence Optimization

9. Performance Optimization (2/2)

- Efficient sequence design
- Sequence reuse
- Sequence caching
- Sequence performance

10. Advanced Debugging Techniques (1/2)

- UVM Debugging
- UVM debugging tools
- Phase debugging
- Component debugging
- Transaction debugging
- Coverage Debugging

10. Advanced Debugging Techniques (2/2)

- Coverage gaps
- Coverage analysis
- Coverage improvement
- Coverage tools

Hands-on examples

Module 5 self-check

```
$ ./scripts/module5.sh --check
```

Terminal: module5_check

```
[[0;36m=====[[0m
[[0;36mModule 5: Advanced UVM Concepts[[0m
[[0;36m=====[[0m
```

Usage: ./scripts/module5.sh [OPTIONS]

Module 5: Advanced UVM Concepts
This script runs examples and tests for Module 5.

OPTIONS:

Examples:

--virtual-sequences Run virtual sequence examples
--coverage Run coverage model examples
--configuration Run configuration object examples
--callbacks Run callback examples
--register-model Run register model examples
--all-examples Run all examples (default)
--skip-examples Skip all examples

Tests:

--pyuvvm-tests Run pyuvvm tests

Environment:

--venv DIR Virtual environment directory (default: .venv)
--no-venv Don't use virtual environment
--sim SIMULATOR Simulator to use (default: verilator)

Other:

--help, -h Show this help message

EXAMPLES:

Run all examples
./scripts/module5.sh

Run specific examples

Exercise scaffold

```
./scripts/module5.sh --scaffold
```

Demo: Virtual Sequences

```
$ .venv/bin/python  
    module5/examples/virtual_sequences/virtual_sequence_example.py
```

Terminal: ex01_virtual_sequences

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module5/examples/virtual_
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Coverage Models

```
$ .venv/bin/python module5/examples/coverage/coverage_example.py
```

Terminal: ex02_coverage

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module5/examples/coverage
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Configuration Objects

```
$ .venv/bin/python  
    module5/examples/configuration/configuration_example.py
```

Terminal: ex03_configuration

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module5/examples/configur
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: UVM Callbacks

```
$ .venv/bin/python module5/examples/callbacks/callback_example.py
```

Terminal: ex04_callbacks

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module5/examples/callback
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Register Model

```
$ .venv/bin/python  
    module5/examples/register_model/register_model_example.py
```

Terminal: ex05_register_model

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module5/examples/register
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Practice & assessment

Exercises

- Virtual Sequences
- Coverage Implementation
- Configuration Design
- Callback Implementation
- Register Model

Assessment checklist

- Can create virtual sequences
- Can implement coverage models
- Can design configuration objects
- Can use callbacks effectively
- Can use advanced register model
- Can coordinate multiple sequencers

Summary & next steps

- Master sequences, coverage, configuration, and virtual sequences
- Complete module5/CHECKLIST.md
- Run `./scripts/module5.sh --check`
- Next: Next module in course